Object Oriented Programming (CS1002) | Spring 2026

Assignment: 03 Total marks: 10

- Read the problem statement carefully and understand what is being asked. Failure to comprehend the problem statement may result in incorrect assumptions and answers.
- **Plagiarism will not be tolerated. Any evidence of copied work, including minor instances, will result in a grade of zero for the entire assignment.**
- Late submissions will not be accepted under any circumstances. Submit your work on time to avoid penalties.
- Submit c++ file on gcr
- If you have any questions or concerns, ask before submitting your work. Lame excuses or attempts to justify plagiarism or late submissions will not be accepted.

Problems :

The Tale of Dynamic Arrays: The Vector Family

Once upon a time in the world of programming, there lived a family of classes known as the Vector Family. This family was renowned for its ability to manage collections of numbers efficiently.

Chapter 1: The Birth of Vector

At the heart of the family was Vector, a wise and strong class that held an array of integers in its capable arms. Vector had a special ability to grow when needed, thanks to its method called resize. Whenever it ran out of space due to an influx of numbers, Vector would double its capacity and safely transfer the existing numbers to a new, larger array.

Vector also had a gentle touch, allowing anyone to add numbers through its push_back method, retrieve numbers with get, and find out how many numbers were currently held with find_len. All the inhabitants of the programming world came to respect Vector for its flexibility and reliability.

Chapter 2: The Uniqueness of UniqueVector

One day, a new member joined the Vector family, known as UniqueVector. UniqueVector admired Vector's abilities but wished to add a twist of its own. It wanted to ensure that only unique numbers could join the collection.

UniqueVector employed its own version of the push_back method, which checked whether a number already existed before adding it to the collection. If it found a duplicate, it would simply refuse to add it, keeping the collection pure and unique. The townsfolk appreciated UniqueVector for maintaining order and preventing clutter in their number collections.

Chapter 3: The Quest of FrequencyVector

Not long after, another relative named FrequencyVector appeared, eager to keep track of how often each number was added to the family. FrequencyVector inherited from Vector but brought along a special array to hold the frequencies of each number added.

Whenever a number was added through FrequencyVector's push_back method, it not only added the number but also updated its frequency count. If a number was new, it would set its frequency to one, while existing numbers would have their frequency increased. This way, FrequencyVector became the go-to class for anyone wanting to know how many times a specific number appeared in the collection.

Your Task : implement these three vector classes in c++ using opp.

Input :

The input format consists of several lines: the first line contains an integer x, indicating the number of elements to be added to the initial Vector. The second line includes space-separated integers A1,A2,…,Ax that represent the values to be inserted into the Vector. The third line contains an integer s representing the number of unique elements to be added to the UniqueVector. The fourth line lists sss space-separated integers B1,B2,…,BsB_1, B_2, for the unique values. The fifth line has an integer f indicating how many elements will be added to the FrequencyVector. The sixth line provides f space-separated integers C1,C2,…,Cf for the values to be inserted into the FrequencyVector. Finally, the seventh line contains three space-separated integers D,E,F whose frequencies will be queried from the FrequencyVector

Output :

1. The first line contains the elements of the main vector.
2. The second line contains the elements of the unique vector.
3. The third line contains the elements of the frequency vector.
4. The fourth line contains the frequency of specific values A, B, and C.

5 1 2 3 4 5 4 1 2 4 4 5 1 2 3 4 5 1 2 3	Printing vector : 1 2 3 4 5 Printing unique vector : 1 2 4 Printing the frequency vector : 1 2 3 4 5 1 1 1

0 0 0 1 2 3	Printing vector : Printing unique vector : Printing the frequency vector : 0 0 0